

Case Study no. 31

HireSense - Applicant Tracking System with a Bias Audit Requirement

Problem Statement:

A staffing company screens 90,000 applications a year. Its current tool ranks candidates and the recruiters trust the ranking, which is exactly the problem: nobody can explain why candidate 412 ranked above candidate 87, and an internal review found the ranking correlated with the applicant's university tier far more strongly than with later job performance. The rebuild must make ranking criteria explicit and configurable, log every automated rejection with a reason, and support a periodic bias audit. There is also a mundane engineering problem underneath: résumé parsing works on 71% of documents and fails silently on the rest. ***(With Proper Justification)***

Given Project Data (use these figures - do not invent your own):

- Ranking routine: if mandatory skills missing, reject; else score experience band; if score $\geq$ threshold and location matches, shortlist; else if score $\geq$ threshold and remote allowed, shortlist; else waitlist; if parse confidence < 0.6 , route to manual review. Draw the CFG and compute cyclomatic complexity.
- Parsing quality: 29% failure rate. Define an acceptance threshold and the effort to reach it as a work package.
- Audit requirement: selection rate for any declared group must not fall below 80% of the highest group's rate. Turn this into a testable requirement.

Objectives

- **Testing & Quality Assurance:** Build the CFG above, compute cyclomatic complexity, derive independent paths, and design the test set including the silent-parse-failure path.
- **Requirements Engineering:** Specify explainability, configurable criteria, audit logging and the fairness threshold as measurable requirements.
- **Software Design & UML:** Separate parsing, scoring and ranking into cohesive modules; draw the class diagram and an activity diagram for the screening pipeline.
- **Metrics & Quality Assurance:** Define the audit metrics, their measurement frequency, and the action triggered by a breach.
- **Project Management:** Plan the rebuild incrementally so screening never stops; produce a WBS, Gantt and risk register including reputational risk.
- **Software Maintenance:** Explain why a fairness audit is preventive maintenance and design

the recurring process around it.

Outcomes

- A cyclomatic complexity value with the complete independent path set and tests.
- A test that specifically catches silent parse failure, which is the current system's real defect.
- Fairness expressed as a number with a defined consequence when breached.
- A modular design where changing a ranking rule does not touch parsing.
- A recurring audit process rather than a one-off report.

Deliverables

- **White-Box Test Report** - CFG, cyclomatic complexity, independent paths, coverage evidence.
- **SRS** - Explainability, configurability, logging and fairness requirements with thresholds.
- **Design Pack** - Class diagram, activity diagram, module cohesion justification.
- **Audit Specification** - Metrics, frequency, thresholds, breach response procedure.
- **Project Plan** - Incremental WBS, Gantt, risk register including reputational risk.
- **Maintenance Process** - Recurring audit cycle defined as preventive maintenance.